

Task-1A

1. Task 1A-a:Autonomous Arena Analysis using OpenCV

The goal of this task is to use Python and OpenCV to:

- perform image manipulation,
- detect colors,
- identify object positions,
- and extract proper coordinates from an image.

This task simulates a simplified computer vision pipeline used in autonomous robotics systems.

Problem Statement

Clone this GitHub repo:[GitHub](#) in your system.

NOTE: Do not change the repository structure in your local system.

The robotics division of Cyborg is developing an autonomous rescue robot for a national robotics challenge.

The robot operates inside a grid-based arena representing a disaster-hit industrial zone. Different colored regions inside the arena represent different environmental conditions that directly affect the robot's navigation and survival.

Before autonomous navigation begins, the robot must first analyze the arena using its onboard vision system.

Each colored block inside the arena has a unique meaning:

Color	Meaning
Red	DANGER
Green	SAFE
Blue	REFUEL
Orange	SLOW
Yellow "S"	Start Position

Cyan "G"	Goal Position
----------	------------------

At two places S and G have been mentioned with yellow and cyan colour. Those squares indicate the start and goal position for the bot. Black and white boxes carry no significance.

Your task is to develop the robot's vision module using OpenCV so that it can:

1. Analyze the arena image,
2. Detect all special colored blocks,
3. Determine their exact grid coordinates,
4. Identify arena dimensions,
5. And generate structured arena information for future path-planning algorithms.

Arena Description

Each image inside the `Test_images` folder contains:

- a square grid-based arena,
- consisting of equal-sized cells,
- containing special navigation zones.

The arena size is NOT fixed and may vary across test cases.

Possible arena dimensions include:

- 6×6
- 8×8
- 10×10
- 12×12

Coordinate System

The arena follows an alphanumeric coordinate system:

- Columns → A, B, C...
- Rows → 1, 2, 3...

Coordinate convention:

- A1 → bottom-left cell
- Top-left cell → depends on arena size
- Bottom-right cell → depends on arena size

Example:

For an 8×8 arena:

- top-left → A8
- bottom-right → H1

Task Requirement

Your program must:

1. Read the input image.
2. Determine the arena dimensions.
3. Detect the Start cell and Goal cell.
4. Identify:
 - the type of each detected cell,
 - and its exact grid coordinate.
5. Return/store the detected arena information properly.

Detection should be performed by analyzing the center region of each cell.

Required Output Dictionary Format (Task 1A-a)

The final output returned by your function must strictly follow the dictionary format shown below.

- Keys must remain exactly the same.
- Cell names must follow these names (A1, B3, H8, etc.).
- special_cells must itself be a dictionary.
- Special cell labels must be one of: DANGER, SAFE, REFUEL, SLOW.

Example Output Format:

```
{ "arena_size": 8,
  "start": "E6",
  "goal": "E7",
  "special_cells": { "A5": "DANGER", "A8": "DANGER", "B7": "SAFE", "C1":
    "SLOW", "C2": "REFUEL", "C8": "SLOW", "D8": "REFUEL", "E4": "SAFE",
    "F3": "SLOW", "F4": "SLOW", "G4": "SAFE", "H1": "REFUEL", "H3": "SLOW"
  }
}
```

Your evaluation depends on the exact structure of this dictionary.

Implementation and Evaluation

Write your logic in the [script.py](#) file at the designated place given in the code.

To test your code, run the [main.py](#) file, it will produce the output as per your logic for arena_1.png. Check if it matches your actual output.

IMPORTANT:

You are only required to correctly populate the `result` dictionary.

The ordering of `special_cells` will automatically be handled by the provided template code.

Do NOT modify:

- function name,
- dictionary structure,
- or return format.

Only write your detection logic inside the designated section of `script.py`.

Once you have tested your `script.py`, run the following command in the terminal from inside the task folder `Task_1A-a`:

`./evaluator`

NOTE: Do not tamper the evaluator file, as it may throw errors later on.

The evaluator will automatically test your implementation on multiple hidden test cases and display the final score.

If your implementation passes the required test cases, the evaluator will generate an encrypted submission file with your roll number as file name.

Task 1A-b : Autonomous Target Tracking using OpenCV

Aim

The goal of this task is to use Python and OpenCV to:

- process video frames,
- perform color-based object detection,
- use contour detection,
- track moving objects,
- and analyze object motion inside an arena.

Problem Statement

Clone this GitHub repo: [GitHub](#) in your system.

NOTE: Do not change the repository structure in your local system.

The robotics division of Cyborg is developing an autonomous surveillance robot for a smart warehouse system.

Inside the warehouse arena, multiple colored payloads move continuously in random directions. The robot's onboard vision system must identify and monitor a specific target payload throughout the operation.

Your task is to design a vision system capable of detecting and tracking the **GREEN** payload while ignoring all other moving objects.

The robot must also analyze the movement pattern of the target payload and determine how many times it collides with each arena wall.

Arena Description

Each test video contains:

- a rectangular arena,
- four moving colored objects:
 - Red
 - Blue
 - Yellow
 - Green

All objects move continuously and bounce off the arena walls.

The GREEN object is the target payload.

The arena background remains static throughout the video.

Task Requirements

Your program must:

1. Read the input video.
2. Detect the GREEN object using OpenCV.
3. Track the object continuously throughout the video.
4. Ignore all non-target objects.
5. Detect collisions of the GREEN object with:
 - top wall,
 - bottom wall,
 - left wall,
 - right wall.
6. Return the total collision count for each wall.

Important Notes

- Detection must be performed using:
 - HSV masking,
 - contour detection,
 - centroid extraction.
- Students are expected to:
 - identify the largest valid contour,
 - compute object center coordinates,
 - and analyze object movement frame-by-frame.
 - Count the total wall strikes on up,down,left,right walls
- Minor variations in HSV ranges are acceptable.

Expected Output Format

Your function must return a dictionary in the following format:

```
{  
    "top_wall_hits": 1,
```

```
"bottom_wall_hits": 2,  
"left_wall_hits": 3,  
"right_wall_hits": 4  
}
```

Function to Implement

Implement the following function inside `script.py`:

`analyze_video(video_path)`

The function should:

- process the video,
- detect the target object,
- and return the final collision counts.

Testing

To test your implementation on a sample video:

`python3 main.py` or run `main.py` script

It will open the video with live counting of wall strikes.

Evaluator

Run the evaluator binary to test your implementation on all test videos.

Execute the following command in the terminal from inside the `Task_1A-b` directory:

`./evaluator`

If your implementation passes the required test cases, the evaluator will generate an encrypted submission file with your roll number as file name.

DEADLINE: 19th May EOD